SALES DATA ENGINEERING AND QUERY OPTIMIZATION USING SNOWFLAKE

STEP 1: SNOWFLAKE ENVIRONMENT SETUP

1.1 Objective

The objective of this step is to create a new Snowflake environment for implementing a sales data engineering and analytical workflow.

The environment consists of a database, separate schemas for raw and cleaned datasets, and a virtual warehouse for executing SQL queries.

The resources are created with consistent naming conventions to organize the sales datasets and support subsequent data ingestion, cleaning, transformation, analysis, and query optimization activities.

1.2 Database Creation

A new database named SALES_ANALYTICS_DB is created to store all the database objects required for the sales analytics project.

SQL Command:

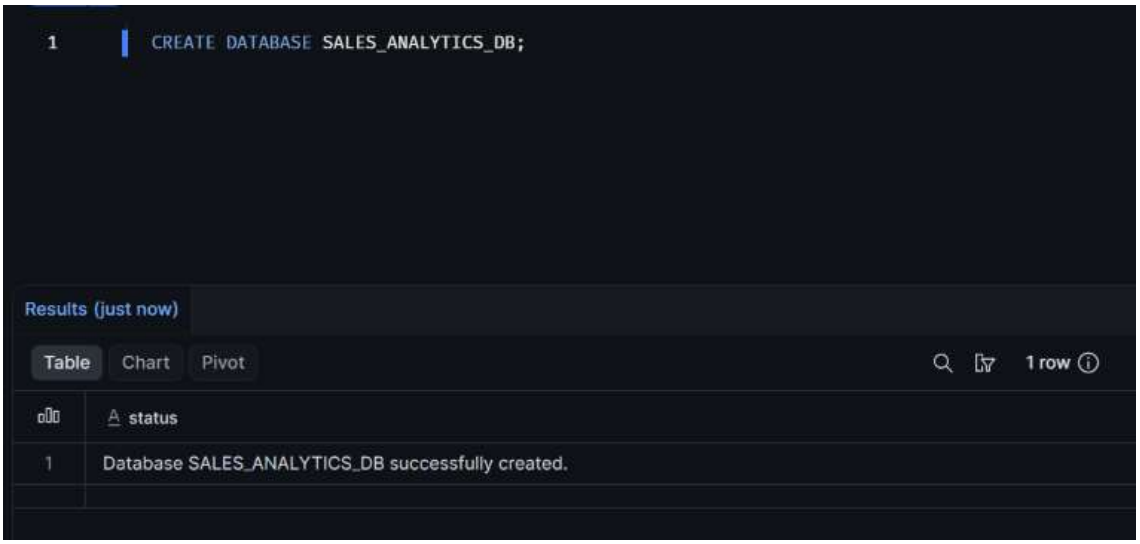
```
CREATE DATABASE SALES_ANALYTICS_DB;
```

Explanation:

- CREATE DATABASE creates a new database in Snowflake.
- SALES_ANALYTICS_DB is the name assigned to the database.

The database acts as the main container for the schemas and tables used in this project.

Screenshot 1: Database Creation



1.3 Schema Creation

Two schemas are created inside the database to organize the datasets based on their processing stages.

SQL Commands:

```
CREATE SCHEMA SALES_ANALYTICS_DB.SALES_RAW;


CREATE SCHEMA SALES_ANALYTICS_DB.SALES_CLEAN;
```

Explanation:

SALES_RAW:

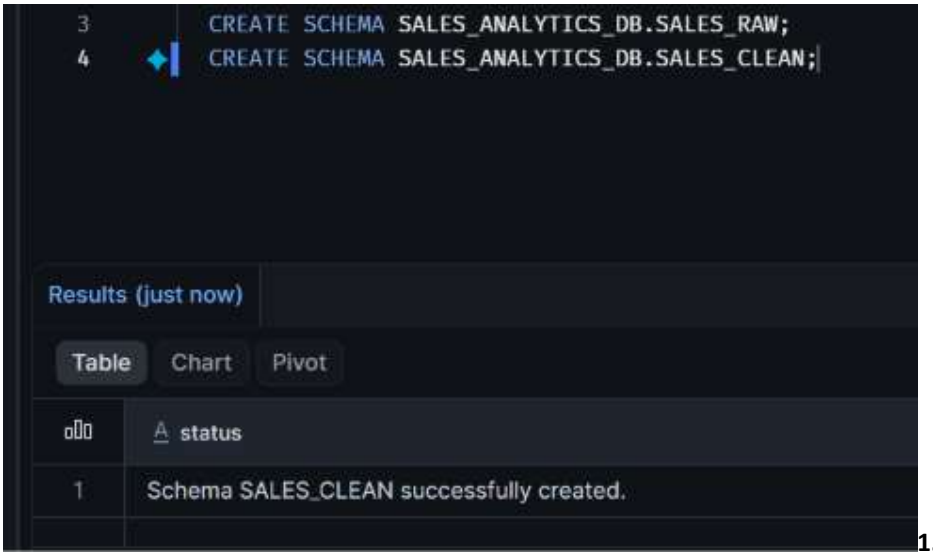
This schema stores the original datasets ingested from CSV files. The raw data is preserved to maintain the original source information.

SALES_CLEAN:

This schema stores cleaned and transformed datasets that are prepared for analytical operations.

Separating raw and cleaned data helps maintain data organization and prevents modifications to the original datasets during transformation.

Screenshot 2: Schema Creation



1.4 Virtual Warehouse Creation

A virtual warehouse named SALES_ANALYTICS_WH is created to provide the computational resources required to execute SQL queries and perform data processing operations.

SQL Command:

```
CREATE WAREHOUSE SALES_ANALYTICS_WH
WITH
   WAREHOUSE_SIZE = 'XSMALL'
   AUTO_SUSPEND = 60
   AUTO_RESUME = TRUE
   INITIALLY_SUSPENDED = TRUE;
```

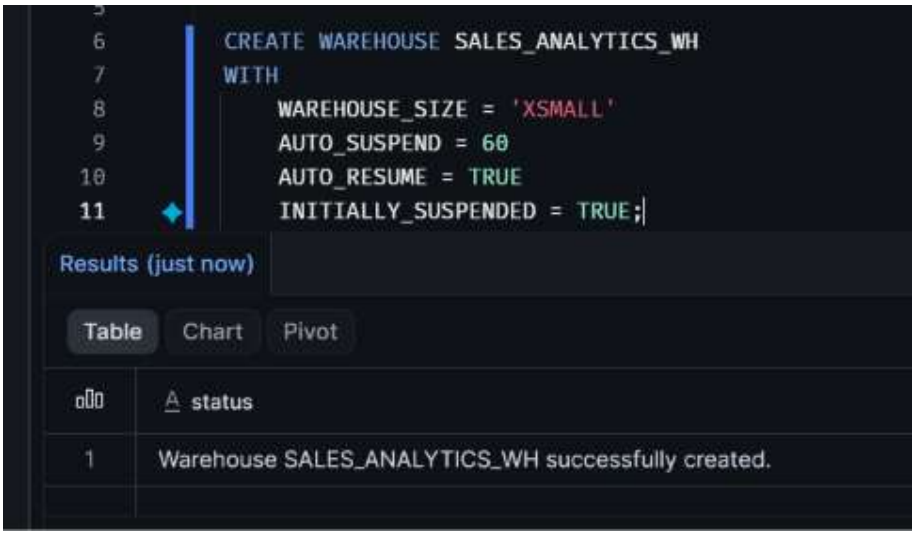
Explanation of Parameters:

Parameter	Description
WAREHOUSE_SIZE	Specifies the compute size of the warehouse. XSMALL is selected for this practical.
AUTO_SUSPEND	Automatically suspends the warehouse after 60 seconds of inactivity.
AUTO_RESUME	Automatically resumes the warehouse when a query requires it.
INITIALLY_SUSPENDED	Creates the warehouse in a suspended state.

The XSMALL warehouse is selected because the project uses relatively small practice datasets.

Automatic suspension and resumption help manage warehouse usage.

Screenshot 3: Virtual Warehouse Creation



1.5 Selecting the Database, Schema and Warehouse

After creating the required resources, the database, schema, and warehouse are selected for subsequent operations.

SQL Commands:

```
USE WAREHOUSE SALES_ANALYTICS_WH;


USE DATABASE SALES_ANALYTICS_DB;


USE SCHEMA SALES_RAW;
```

Explanation:

- USE WAREHOUSE selects the virtual warehouse that will execute SQL queries.

- USE DATABASE selects the database in which the project operations will be performed.
- USE SCHEMA selects the raw schema as the current working schema.

These commands establish the working context for subsequent database operations.

1.6 Environment Verification

The current Snowflake configuration is verified using the following SQL query.

SQL Command:

```
SELECT
  CURRENT_DATABASE() AS DATABASE_NAME,
  CURRENT_SCHEMA() AS SCHEMA_NAME,
  CURRENT_WAREHOUSE() AS WAREHOUSE_NAME,
  CURRENT_ROLE() AS ROLE_NAME;
```

Explanation:

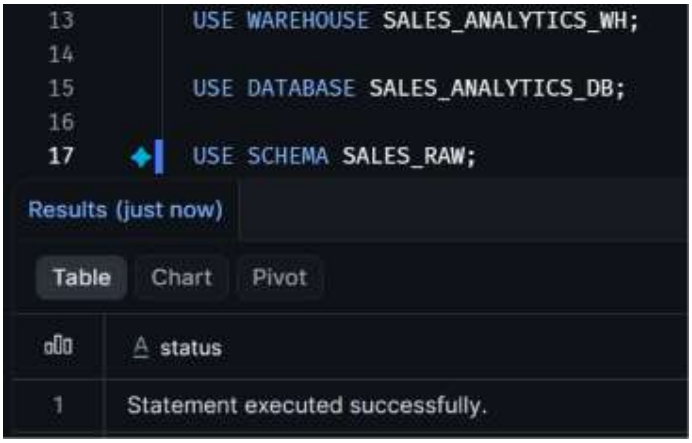
The query retrieves the currently selected database, schema, warehouse, and active user role.

This verification ensures that the required resources have been selected correctly before proceeding with data ingestion.

Expected Configuration:

Property	Expected Value
DATABASE_NAME	SALES_ANALYTICS_DB
SCHEMA_NAME	SALES_RAW
WAREHOUSE_NAME	SALES_ANALYTICS_WH
ROLE_NAME	Active Snowflake role

Screenshot 4: Environment Verification



1.7 Result

The Snowflake environment setup is completed by creating the SALES_ANALYTICS_DB database, SALES_RAW and SALES_CLEAN schemas, and the SALES_ANALYTICS_WH virtual warehouse.

The required database resources are selected and verified successfully.

The environment is now ready for the next stage of the practical, which involves identifying, inspecting, and preparing multiple related sales datasets for ingestion into Snowflake.

STEP 2: DATASET IDENTIFICATION AND PREPARATION

2.1 Dataset Selection

Three related CSV datasets were selected for the sales analytics project:

1. **Customers:** Contains customer information such as CustomerID, CustomerName, Email, City, Country and JoinDate.
2. **Products:** Contains product information such as ProductID, ProductName, Category, UnitCost and ListPrice.
3. **Sales:** Contains transaction information such as SaleID, OrderDate, CustomerID, ProductID, Quantity, UnitPrice, Discount and PaymentStatus.

2.2 Dataset Relationships

The datasets contain the following relationships:

- CustomerID connects the Customers and Sales datasets.
- ProductID connects the Products and Sales datasets.

These relationships will be used to perform joins and analytical operations.

2.3 Dataset Inspection

The CSV files were inspected to understand their structure, column names, data types and record counts.

Dataset	Total Records
Customers	21
Products	13
Sales	61

2.4 Data Quality Observations

During initial inspection, the following potential data-quality issues were identified:

- Duplicate records.
- Missing and NULL values.

- Inconsistent text formatting and whitespace.
- Invalid and inconsistent date formats.
- Invalid numeric values.
- Invalid customer and product references.
- Inconsistent payment status values.

These issues will be addressed during the data validation and cleaning stages.

2.5 Result

Three related sales datasets were successfully prepared and inspected. Their structures, relationships and potential data-quality issues were identified for subsequent Snowflake ingestion and transformation.

STEP 3: TABLE CREATION, FILE FORMAT AND INTERNAL STAGE

3.1 Table Creation

Three tables were created in the SALES_RAW schema:

- CUSTOMER_DATA – Stores customer information.
- PRODUCT_DATA – Stores product information.
- SALES_TRANSACTIONS – Stores sales transaction records.

VARCHAR columns were initially used to preserve raw CSV values for subsequent validation and transformation.

3.2 CSV File Format Creation

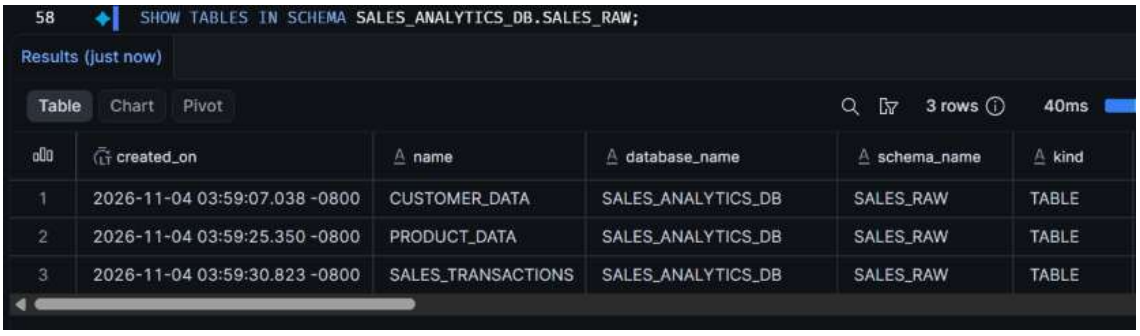
A reusable CSV file format named SALES_CSV_FORMAT was created with a comma delimiter, header skipping, optional quotation marks and empty-field handling.

3.3 Internal Stage Creation

An internal stage named SALES_DATA_STAGE was created to store the CSV files before loading them into the corresponding Snowflake tables.

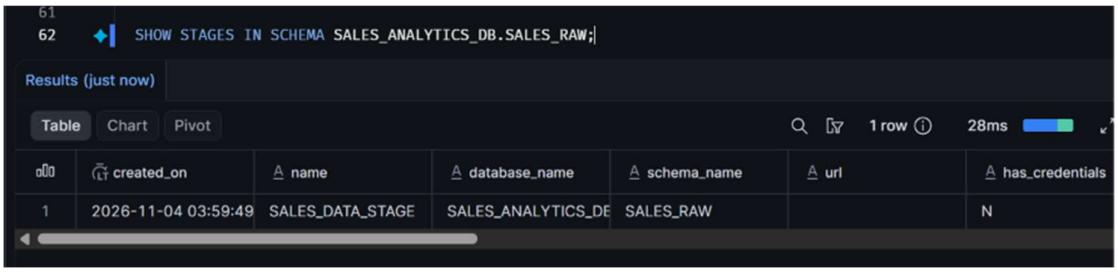
3.4 Verification

The SHOW TABLES, SHOW FILE FORMATS and SHOW STAGES commands were executed to verify the created database objects.



	created_on	name	database_name	schema_name	kind
1	2026-11-04 03:59:07.038 -0800	CUSTOMER_DATA	SALES_ANALYTICS_DB	SALES_RAW	TABLE
2	2026-11-04 03:59:25.350 -0800	PRODUCT_DATA	SALES_ANALYTICS_DB	SALES_RAW	TABLE
3	2026-11-04 03:59:30.823 -0800	SALES_TRANSACTIONS	SALES_ANALYTICS_DB	SALES_RAW	TABLE





3.5 Result

Three raw tables, one CSV file format and one internal stage were created successfully in the SALES_RAW schema.

STEP 4: CSV UPLOAD AND DATA INGESTION

4.1 CSV File Upload

The customers.csv, products.csv and sales.csv files were uploaded to the Snowflake internal stage SALES_DATA_STAGE using Snowsight.

4.2 File Verification

The LIST command was executed to verify the uploaded files in the internal stage.

4.3 Data Loading

The COPY INTO command was used to load the three CSV files into their corresponding raw tables:

- customers.csv → CUSTOMER_DATA
- products.csv → PRODUCT_DATA
- sales.csv → SALES_TRANSACTIONS

The ON_ERROR option was set to CONTINUE to allow valid records to load while reporting records that could not be processed.

4.4 Record Count Verification

COUNT(*) queries were executed on all three tables to verify the number of successfully loaded records.







4.5 Result

The CSV datasets were uploaded to the internal stage and loaded into their corresponding Snowflake raw tables. Loading results and record counts were inspected.

STEP 5: DATA EXPLORATION AND VALIDATION

5.1 Displaying Records

SELECT * queries were executed on the Customers, Products and Sales tables to inspect the imported records.

5.2 Filtering Records

WHERE conditions were used to retrieve specific records based on payment status.

```
SELECT *
FROM SALES_TRANSACTIONS
WHERE PAYMENT_STATUS = 'Paid';
```

Results (just now)

Table | Chart | Pivot — 15 rows — 44ms

	SALE_ID	ORDER	CUSTOMER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE	DISCOUNT	PAY
	S0... 6.7% S0... 6.7% +13 more	2024-01... 6.7% 2024-02... 6.7% +13 more	C008 13.3% C012 13.3% +10 more	P010 20.0% P001 13.3% +7 more	3 26.7% 5 26.7% +3 more	1199.0 20.0% 149.0 20.0% +6 more	0 33.3% 0.1 26.7% +2 more	Paid
1	S0005	2024-12-17	C018	P009	2	149.0	0.1	Paid
2	S0010	2024-07-03	C008	P009	1	149.0	0.1	Paid
3	S0012	2024-06-12	C010	P006	2	399.0	0.1	Paid
4	S0013	2024-05-19	C019	P010	3	1199.0	0	Paid
5	S0014	2024-08-27	C012	P001	3	799.0	null	Paid
6	S0015	2024-05-12	C007	P005	1	8999.0	0.15	Paid
7	S0020	2024-07-25	C999	P011	4	1999.0	0	Paid
8	S0023	2024-07-27	C008	P999	5	149.0	0.15	Paid

5.3 NULL Value Identification

Queries using IS NULL and TRIM() were executed to identify missing and empty values in important columns.

```
SELECT *
FROM CUSTOMER_DATA
WHERE EMAIL IS NULL OR TRIM(EMAIL) = '';
```

Results (just now)

Table | Chart | Pivot — 1 row — 50ms

	CUSTOMER_ID	CUSTOMER_N	EMAIL	CITY	COUNTRY	JOIN_DATE
1	C003	Kabir Nair	null	Chennai	India	2023-02-18

```
SELECT *
FROM PRODUCT_DATA
WHERE CATEGORY IS NULL OR TRIM(CATEGORY) = ''
   OR UNIT_COST IS NULL OR TRIM(UNIT_COST) = '';
```

Results (just now)

Table | Chart | Pivot — 2 rows — 53ms

	PRODUCT_ID	PRODUCT_NAME	CATEGORY	UNIT_COST	LIST_PRICE
1	P006	Water Bottle	null	180	399
2	P008	Notebook Pack	Stationery	null	199

```
SELECT *
FROM SALES_TRANSACTIONS
WHERE QUANTITY IS NULL OR TRIM(QUANTITY) = ''
   OR UNIT_PRICE IS NULL OR TRIM(UNIT_PRICE) = ''
   OR DISCOUNT IS NULL OR TRIM(DISCOUNT) = '';
```

Results (just now)

Table | Chart | Pivot — 2 rows — 48ms

	SALE_ID	ORDER_DATE	CUSTOMER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE	DISCOUNT
1	S0008	2024-06-30	C019	P001	null	799.0	0.15
2	S0014	2024-08-27	C012	P001	3	799.0	null

5.4 Result

The raw datasets were explored, filtered and inspected. Missing values and columns requiring data type conversion were identified for subsequent cleaning and transformation.

STEP 6: HANDLING NULL VALUES AND DATA TYPE CONVERSION

6.1 NULL Value Handling

Missing values were handled according to the nature of each column.

- Missing customer emails were replaced with 'Not Provided'.
- Missing country values were replaced with 'Unknown'.
- Missing product categories were replaced with 'Uncategorized'.
- Unknown product costs and quantities were retained as NULL.
- Missing sales discounts were treated as zero.

6.2 Data Type Conversion

TRY_TO_DATE, TRY_TO_NUMBER and TRY_TO_DECIMAL functions were used to convert raw string values into appropriate date and numeric data types.

TRIM and INITCAP functions were applied to standardize text values.

6.3 Cleaned Table Creation

Three cleaned tables were created in the SALES_CLEAN schema:

- CLEAN_CUSTOMERS
- CLEAN_PRODUCTS
- CLEAN_SALES

The original raw tables were preserved.

```sql
CREATE TABLE CLEAN_CUSTOMERS AS
SELECT
    TRIM(CUSTOMER_ID) AS CUSTOMER_ID,
    INITCAP(TRIM(CUSTOMER_NAME)) AS CUSTOMER_NAME,
    COALESCE(NULLIF(TRIM(EMAIL), ''), 'Not Provided') AS EMAIL,
    INITCAP(TRIM(CITY)) AS CITY,
    COALESCE(NULLIF(TRIM(COUNTRY), ''), 'Unknown') AS COUNTRY,
    TRY_TO_DATE(JOIN_DATE, 'YYYY-MM-DD') AS JOIN_DATE
FROM SALES_RAW.CUSTOMER_DATA;
```

Results (just now)

Table | Chart | Pivot

	status
1	Table CLEAN_CUSTOMERS successfully created.

```sql
SELECT
    TRIM(PRODUCT_ID) AS PRODUCT_ID,
    INITCAP(TRIM(PRODUCT_NAME)) AS PRODUCT_NAME,
    COALESCE(
        NULLIF(INITCAP(TRIM(CATEGORY)), ''),
        'Uncategorized'
    ) AS CATEGORY,
    TRY_TO_DECIMAL(NULLIF(TRIM(UNIT_COST), ''), 10, 2) AS UNIT_COST,
    TRY_TO_DECIMAL(NULLIF(TRIM(LIST_PRICE), ''), 10, 2) AS LIST_PRICE
FROM SALES_RAW.PRODUCT_DATA;
```

Results (just now)

Table | Chart | Pivot

	status
1	Table CLEAN_PRODUCTS successfully created.

```sql
CREATE TABLE CLEAN_SALES AS
SELECT
    TRIM(SALE_ID) AS SALE_ID,
    COALESCE(
        TRY_TO_DATE(ORDER_DATE, 'YYYY-MM-DD'),
        TRY_TO_DATE(ORDER_DATE, 'DD/MM/YYYY')
    ) AS ORDER_DATE,
    TRIM(CUSTOMER_ID) AS CUSTOMER_ID,
    TRIM(PRODUCT_ID) AS PRODUCT_ID,
    TRY_TO_NUMBER(NULLIF(TRIM(QUANTITY), '')) AS QUANTITY,
    TRY_TO_DECIMAL(NULLIF(TRIM(UNIT_PRICE), ''), 10, 2) AS UNIT_PRICE,
    COALESCE(
        TRY_TO_DECIMAL(NULLIF(TRIM(DISCOUNT), ''), 5, 2),
        0
    ) AS DISCOUNT,
    INITCAP(TRIM(PAYMENT_STATUS)) AS PAYMENT_STATUS
FROM SALES_RAW.SALES_TRANSACTIONS;
```

Results (just now)

Table | Chart | Pivot

	status
1	Table CLEAN_SALES successfully created.

6.4 Verification

DESC TABLE commands were used to verify the converted column data types.

```sql
DESC TABLE CLEAN_CUSTOMERS;
```

Results (just now)

Table | Chart | Pivot

	name	type	kind	null?	def
1	CUSTOMER_ID	VARCHAR(10)	COLUMN	Y	null
2	CUSTOMER_NAME	VARCHAR(300)	COLUMN	Y	null
3	EMAIL	VARCHAR(150)	COLUMN	Y	null
4	CITY	VARCHAR(150)	COLUMN	Y	null
5	COUNTRY	VARCHAR(50)	COLUMN	Y	null
6	JOIN_DATE	DATE	COLUMN	Y	null

```
165  DESC TABLE CLEAN_PRODUCTS;
```

Results (just now)

Table | Chart | Pivot

	name	type	kind	null?	default
1	PRODUCT_ID	VARCHAR(10)	COLUMN	Y	null
2	PRODUCT_NAME	VARCHAR(300)	COLUMN	Y	null
3	CATEGORY	VARCHAR(150)	COLUMN	Y	null
4	UNIT_COST	NUMBER(10,2)	COLUMN	Y	null
5	LIST_PRICE	NUMBER(10,2)	COLUMN	Y	null

```
167  DESC TABLE CLEAN_SALES;
```

Results (just now)

Table | Chart | Pivot

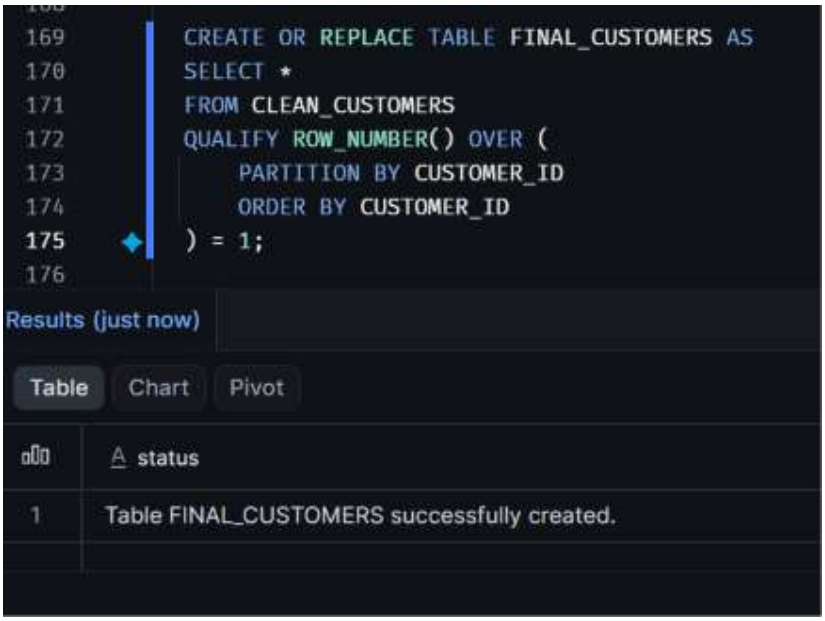
	name	type	kind	null?
1	SALE_ID	VARCHAR(10)	COLUMN	Y
2	ORDER_DATE	DATE	COLUMN	Y
3	CUSTOMER_ID	VARCHAR(10)	COLUMN	Y
4	PRODUCT_ID	VARCHAR(10)	COLUMN	Y
5	QUANTITY	NUMBER(38,0)	COLUMN	Y
6	UNIT_PRICE	NUMBER(10,2)	COLUMN	Y
7	DISCOUNT	NUMBER(5,2)	COLUMN	Y
8	PAYMENT_STAT	VARCHAR(90)	COLUMN	Y

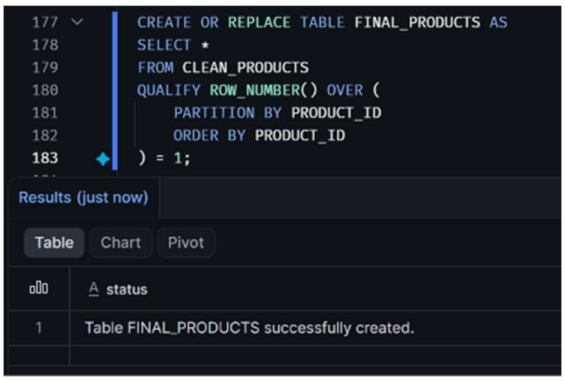
6.5 Result

Cleaned tables were created with standardized text values, handled missing values and converted date and numeric columns. Invalid values requiring further investigation were retained as NULL for subsequent validation.

7.1 Duplicate Record Handling

Duplicate records were identified and removed from the Customers, Products and Sales datasets using ROW_NUMBER() and QUALIFY. This ensured that the final tables contained unique records based on their respective IDs.





STEP 8: ANALYTICAL SQL QUERIES

8.1 Data Filtering

WHERE clauses were used to retrieve and filter records from the final tables.

```sql
SELECT *
FROM FINAL_SALES
WHERE PAYMENT_STATUS = 'Paid';
```

Results (just now)

Table | Chart | Pivot — 14 rows — 135ms

	SALE_ID	ORDER_DATE	CUSTOMER_ID	PRODU	QUANTITY	UNIT_PRICE	DISCOUNT	PAY
	S0... 7.1% S0... 7.1% +12 more	1/15/2... 12/24/...	C012 14.3% C018 14.3% +9 more	P010 21.4% P001 14.3% +7 more	1 5	149 8999	0 0.15	Paid
1	S0034	2024-06-03	C016	P012	5	1499.00	0.05	Paid
2	S0033	2024-12-24	C017	P011	1	1999.00	0.00	Paid
3	S0005	2024-12-17	C018	P009	2	149.00	0.10	Paid
4	S0015	2024-05-12	C007	P005	1	8999.00	0.15	Paid
5	S0012	2024-06-12	C010	P006	2	399.00	0.10	Paid
6	S0017	2024-04-03	C019	P007	2	1299.00	0.10	Pai
7	S0040	2024-09-10	C018	P004	3	6999.00	0.15	Paid
8	S0014	2024-08-27	C012	P001	3	799.00	0.00	Paid

8.2 Total Sales Calculation

SUM() was used to calculate total sales revenue from paid transactions after applying discounts.

```sql
SELECT
    SUM(QUANTITY * UNIT_PRICE * (1 - DISCOUNT)) AS TOTAL_SALES
FROM FINAL_SALES
WHERE PAYMENT_STATUS = 'Paid';
```

Results (just now)

Table | Chart | Pivot

	TOTAL_SALES
1	73754.5000

8.3 Product-wise and Customer-wise Analysis

JOIN, GROUP BY and aggregate functions were used to calculate sales quantities and revenue for individual products and customers.

```sql
SELECT
    P.PRODUCT_NAME,
    SUM(S.QUANTITY) AS TOTAL_QUANTITY,
    SUM(S.QUANTITY * S.UNIT_PRICE * (1 - S.DISCOUNT)) AS TOTAL_SALES
FROM FINAL_SALES S
JOIN FINAL_PRODUCTS P
    ON S.PRODUCT_ID = P.PRODUCT_ID
WHERE S.PAYMENT_STATUS = 'Paid'
GROUP BY P.PRODUCT_NAME
ORDER BY TOTAL_SALES DESC;
```

Results (just now)

Table | Chart | Pivot — 9 rows — 178ms

	PRODUCT_NAME	TOTAL_QUANTITY	TOTAL_SALES
1	Study Desk	3	23847.3500
2	Office Chair	3	17847.4500
3	Led Desk Lamp	13	14687.7500
4	Monitor Stand	5	7120.2500
5	Wireless Mouse	6	4794.0000
6	Backpack	2	2338.2000
7	Bluetooth Speaker	1	1999.0000
8	Water Bottle	2	718.20
9	Gel Pen Set	3	402.3000

```sql
SELECT
    C.CUSTOMER_NAME,
    SUM(S.QUANTITY * S.UNIT_PRICE * (1 - S.DISCOUNT)) AS TOTAL_SALES
FROM FINAL_SALES S
JOIN FINAL_CUSTOMERS C
    ON S.CUSTOMER_ID = C.CUSTOMER_ID
WHERE S.PAYMENT_STATUS = 'Paid'
GROUP BY C.CUSTOMER_NAME
ORDER BY TOTAL_SALES DESC;
```

Results (just now)

Table | Chart | Pivot — 11 rows — 148ms

	CUSTOMER_NAME	TOTAL_SALES
	Aadhya Singh 9.1% Advik Menon 9.1% +9 more	134.1 … 18115.65
1	Tara Kulkarni	18115.6500
2	Kabir Nair	16198.2000
3	Arjun Rao	7649.1500
4	Myra Joshi	7492.7500
5	Ira Choudhary	7120.2500
6	Krish Malhotra	5995.0000
7	Atharv Pillai	5935.2000
8	Advik Menon	2397.0000

8.4 Combining Multiple Tables

The Customers, Products and Sales tables were joined using their common IDs to retrieve combined sales information.

```sql
SELECT
    S.SALE_ID,
    C.CUSTOMER_NAME,
    P.PRODUCT_NAME,
    S.ORDER_DATE,
    S.QUANTITY,
    S.UNIT_PRICE,
    S.DISCOUNT,
    S.PAYMENT_STATUS
FROM FINAL_SALES S
JOIN FINAL_CUSTOMERS C
    ON S.CUSTOMER_ID = C.CUSTOMER_ID
JOIN FINAL_PRODUCTS P
    ON S.PRODUCT_ID = P.PRODUCT_ID;
```

Results (just now)

Table | Chart | Pivot — 51 rows — 91ms

	SALE_ID	CUSTOMER_NAME	PRODUCT_NAME	ORDER_DATE	QUANTITY	UNIT_PRICE	DISCOUNT	PAYMENT_STATUS
	S0001 2.0% S0002 2.0% +49 more	Tara Kulkarni 11.8% Advik Menon 7.8% +17 more	Backpack 11.8% Office Chair 11.8% +10 more	1/15/2024 12/24/2...	1 5	149 8999	0 0.15	Paid 27.5% Refunded 27.5% +2 more
1	S0044	Tara Kulkarni	Usb-C Hub	2024-03-25	5	1499.00	0.10	Refunded
2	S0057	Advik Menon	Backpack	2024-11-08	3	1299.00	0.10	Refunded
3	S0003	Advik Menon	Monitor Stand	2024-08-02	3	1499.00	0.10	Refunded
4	S0034	Ira Choudhary	Monitor Stand	2024-06-03	5	1499.00	0.05	Paid
5	S0052	Advik Menon	Water Bottle	2024-10-21	5	399.00	0.05	Refunded
6	S0002	Tara Kulkarni	Bluetooth Speaker	2024-05-22	1	1999.00	0.00	Pending
7	S0037	Tara Kulkarni	Study Desk	2024-05-17	3	8999.00	0.10	Refunded
8	S0043	Kabir Nair	Backpack	2024-08-12	5	1299.00	0.15	Pending
9	S0059	Arjun Rao	Office Chair	2024-11-20	4	6999.00	0.00	Refunded
10	S0027	Myra Joshi	Usb-C Hub	2024-04-18	3	1499.00	0.15	Cancelled
11	S0033	Rudra Bose	Bluetooth Speaker	2024-12-24	1	1999.00	0.00	Paid
12	S0039	Aadhya Singh	Water Bottle	2024-06-13	1	399.00	0.10	Pending

Feedback

8.5 Result

Analytical SQL queries were executed to retrieve, filter and analyze sales data across multiple related tables.

STEP 9: SQL QUERY OPTIMIZATION

9.1 Original Query

A product-wise sales query was executed using JOIN, WHERE and GROUP BY operations to calculate total sales for each product.

```sql
SELECT
    P.PRODUCT_NAME,
    SUM(S.QUANTITY * S.UNIT_PRICE * (1 - S.DISCOUNT)) AS TOTAL_SALES
FROM FINAL_SALES S
INNER JOIN FINAL_PRODUCTS P
    ON S.PRODUCT_ID = P.PRODUCT_ID
WHERE S.PAYMENT_STATUS = 'Paid'
GROUP BY P.PRODUCT_NAME
ORDER BY TOTAL_SALES DESC;
```

Results (just now)

Table | Chart | Pivot — 9 rows — 569ms

	PRODUCT_NAME	TOTAL_SALES
1	Study Desk	23847.3500
2	Office Chair	17847.4500
3	Led Desk Lamp	14687.7500
4	Monitor Stand	7120.2500
5	Wireless Mouse	4794.0000
6	Backpack	2338.2000
7	Bluetooth Speaker	1999.000
8	Water Bottle	718.2
9	Gel Pen Set	402.3000

9.2 Optimized Query

The query was rewritten using a CTE to filter paid transactions and aggregate sales before joining the product table.

```sql
WITH PRODUCT_SALES AS (
    SELECT
        PRODUCT_ID,
        SUM(QUANTITY * UNIT_PRICE * (1 - DISCOUNT)) AS TOTAL_SALES
    FROM FINAL_SALES
    WHERE PAYMENT_STATUS = 'Paid'
    GROUP BY PRODUCT_ID
)
SELECT
    P.PRODUCT_NAME,
    PS.TOTAL_SALES
FROM PRODUCT_SALES PS
JOIN FINAL_PRODUCTS P
    ON PS.PRODUCT_ID = P.PRODUCT_ID
ORDER BY PS.TOTAL_SALES DESC;
```

Results (just now)

Table | Chart | Pivot — 9 rows — 84ms

	PRODUCT_NAME	TOTAL_SALES
1	Study Desk	23847.3500
2	Office Chair	17847.4500
3	Led Desk Lamp	14687.7500
4	Monitor Stand	7120.2500
5	Wireless Mouse	4794.0000
6	Backpack	2338.2000
7	Bluetooth Speaker	1999.0000
8	Water Bottle	718.2000
9	Gel Pen Set	402.3000

9.3 Performance Comparison

Both queries were executed and their results, execution times and query IDs were recorded for comparison.

9.4 Result

Original and optimized SQL queries were prepared to study the impact of filtering, aggregation and query structure on execution performance.

STEP 10: QUERY PROFILE ANALYSIS

10.1 Query Profile Inspection

Snowflake Query Profile was used to inspect the execution plans of the original and optimized queries.

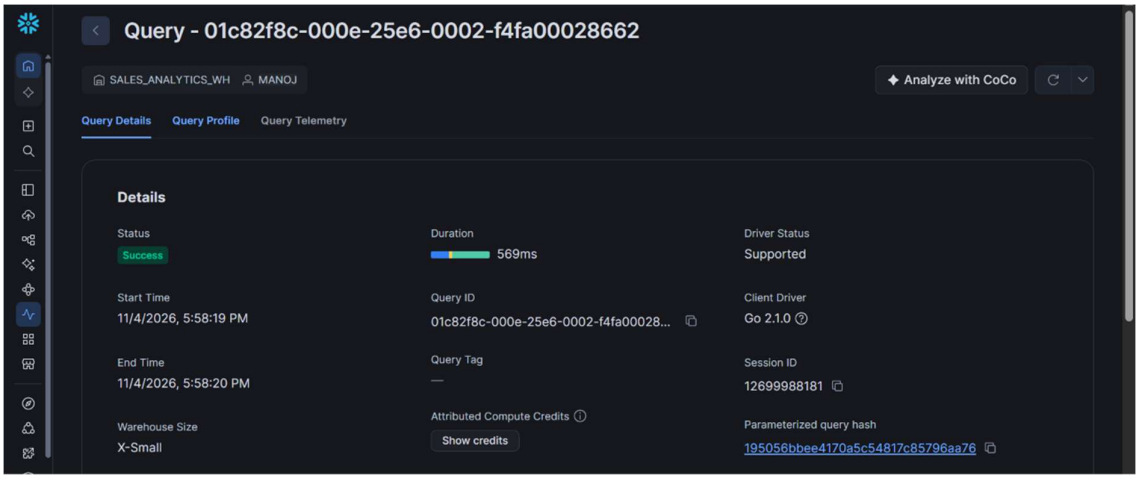
10.2 Performance Metrics

The following details were examined:

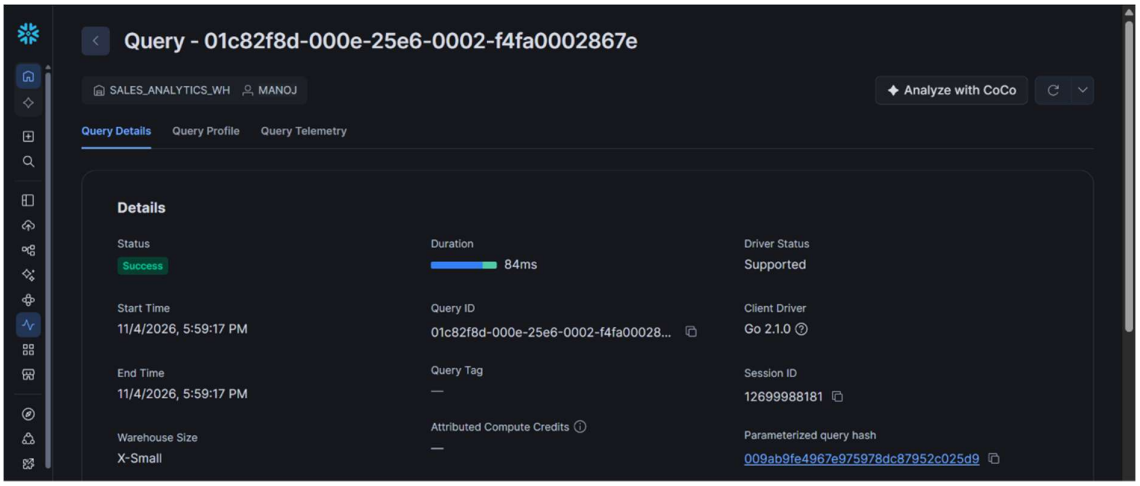
- Query execution time.
- Data scanned.
- Table scan operations.
- Join operations.
- Aggregation operations.
- Other relevant execution statistics.

10.3 Performance Comparison

The execution statistics of both queries were compared to understand the effect of query restructuring and optimization.



GENERAL QUERY



OPTIMIZED QUERY

-> Duration of Optimized query is 8x faster than General query

10.4 Result

Query Profile provided information about query execution and processing operations. The comparison helped evaluate the effectiveness of the applied optimization techniques.